WHITE PAPER

# Verified Iterative Refinement

*A Methodology for Deterministic Software Delivery Using Large Language Models*

| | |
|---|---|
| **Methodology** | Documentation-First, Verification-Driven Development |
| **Domain** | AI-Assisted Software Engineering |
| **Version** | 3.0 — reflects README.md, METHODOLOGY.md, CLAUDE-CODE-GUIDE.md (2026) |
| **Case Result** | Full-stack production system in 4 days / ~20 person-hours each |


## Abstract

Large Language Models (LLMs) are fundamentally probabilistic systems. Every token they generate carries an inherent error rate that cannot be eliminated by prompt engineering alone. This paper proposes and validates a methodology — Verified Iterative Refinement (VIR) — that does not attempt to eliminate this probabilistic nature, but instead engineers around it.

VIR rests on seven core principles: a strict document authority hierarchy with human-authored specifications as inviolable ground truth; status-driven problem tracking; mandatory verification embedded into every development step; iterative cycles with reconciliation; narrative problem solving that creates durable institutional memory; precise cross-references that eliminate ambiguity; and acceptance tests as runnable documentation. Together these principles form a complete documentation and verification discipline optimised for projects built with AI coding agents.

A real-world case study demonstrates that two non-developer practitioners, using Claude Code as the AI agent, delivered a complete production system — including live data ingestion, analysis pipeline, database, frontend, and on-premises deployment — in four days and approximately 20 person-hours each. The paper derives the theoretical basis for this outcome, formalises the methodology as it has evolved in practice, and provides practical guidance for teams adopting VIR with Claude Code.


© 2026 VIR Methodology

# 1. The Problem: Probabilistic Engines in Deterministic Contexts

## 1.1 LLMs as Noisy Generators

Software engineering is a deterministic discipline. A function either produces the correct output or it does not. A system either meets its specification or it fails. Yet the tools now available to generate software — Large Language Models — are fundamentally probabilistic. They do not compute outputs; they sample from distributions over plausible next tokens. Every line of code an LLM writes is, in a precise sense, a hallucination that happens to be useful.

This creates a structural tension: practitioners want to use LLMs for their extraordinary generative speed and breadth, but cannot accept probabilistic correctness guarantees in production systems.

## 1.2 Why Prompt Engineering Alone Fails

The conventional response to LLM unreliability is to improve prompts. Better instructions, more context, clearer examples — all of these reduce the hallucination rate at the margin. But they do not eliminate it, and they do not scale. As a codebase grows, the surface area for error grows with it. A 5% per-generation error rate on a 100-line function is different in kind from a 5% error rate on a 10,000-line system.

What is needed is not better prompting, but a systems-level feedback architecture that bounds and corrects the probabilistic behaviour of the generator.

# 2. Verified Iterative Refinement: The Framework

## 2.1 Core Principle

> *Treat the LLM as a noisy sensor. You do not eliminate the noise — you build a feedback loop that converges on the correct signal.*

VIR draws on an analogy from control theory. A Kalman filter does not require a perfect sensor; it requires a model of sensor noise and a correction mechanism. Similarly, VIR does not require a perfect LLM; it requires a model of where LLM errors tend to occur and a reconciliation mechanism that catches and corrects them before they propagate.

## 2.2 The Seven Core Principles

### P1: Document Authority Hierarchy

All truth flows from a root document — typically a PRD. Extension documents are subordinate and must not contradict the root. On any conflict, the root wins. The full hierarchy:

```
Root PRD  (axiomatic source of truth)
  ├── Extension Specs  (extend but never override root)
  ├── Implementation Plans  (derived from specs, serial execution)
  └── Operational Docs  (bugs, issues, warnings, tests, deployment)
```

- The root PRD is always right — if a sub-document contradicts it, the root wins
- Extension specs must explicitly state which PRD section they extend
- Plans must reference the spec they are derived from
- Operational docs reference specific PRD sections when logging issues

### P2: Status-Driven Tracking

Every documented problem — bug, issue, warning — carries an explicit status reflecting three dimensions:

- Scope — which component or file is affected
- Timeline — when it was discovered, resolved, or superseded
- Confidence — resolved, pending confirmation, mitigated, superseded, or accepted

### P3: Verification-First Development

Implementation plans include embedded acceptance tests in every step. Tests are runnable code (bash/Python), not prose descriptions. Steps are executed serially — each step must verify the previous step's state before proceeding. This is not "write tests after the fact." The tests are written as part of the plan, before any code is written.

### P4: Iterative Cycles with Reconciliation

- Steps are serial, not parallel
- Each step depends on verification of the previous step
- "Do not skip reconciliation" is a core rule
- Completed work is reconciled against specs before moving forward

### P5: Narrative Problem Solving

Every bug, issue, and warning tells a complete story:

```
Discovery → Diagnosis → Root Cause → Fix → Validation → Integration Notes
```

This creates institutional memory that prevents repeat failures and helps AI agents understand not just what was done, but why.

### P6: Precise Cross-References

All cross-links use file paths, line numbers, function names, and section numbers. Generic references ("the code", "somewhere in the handler") are avoided. This enables both human developers and AI agents to locate and verify referenced information without ambiguity.

### P7: Acceptance Tests as Documentation

Tests are runnable code embedded in .md files. They serve as both verification and specification. This eliminates the gap between "what the doc says" and "what the code does." A test that cannot be run is not a test — it is a wish.

## 2.3 The Five-Phase Development Loop

### Phase 1: Specify

1. Write or update the root PRD with the system's requirements
2. Create extension specs for complex subsystems
3. Define success criteria — measurable outcomes that determine when the work is done

### Phase 2: Plan

1. Derive an implementation plan from the spec
2. Break work into serial steps — each with a clear goal, deliverables, and acceptance test
3. Identify reuse opportunities from existing code
4. Include reconciliation checkpoints at the end

### Phase 3: Build

1. Execute steps serially — do not skip ahead
2. Run each step's acceptance test before proceeding
3. If a test fails, fix the issue before moving to the next step
4. If the plan needs to change, update the plan document first

### Phase 4: Reconcile

1. Compare all deliverables against the spec
2. Run the full test suite
3. Record success criteria results (PASS / FAIL with notes)
4. Update CLAUDE.md with new module documentation
5. File any discovered issues in ISSUES.md

### Phase 5: Operate

1. Track production bugs in BUGS.md with full narratives
2. Track known limitations in WARNINGS.md with mitigation status
3. Track feature gaps in ISSUES.md with proposed fixes
4. Update the root PRD when the system's behaviour diverges from the spec

## 2.4 The Reconciliation Loop

For each step in the implementation plan, the following cycle is executed:

| Step | Actor | Action |
|---|---|---|
| 1. Generate | LLM Agent | Develops functionality from PRD + plan step; executes associated tests autonomously |
| 2. Record | LLM Agent | Appends all test results to TESTS.md — persistent, auditable record |
| 3. Reconcile | LLM Agent | Explicitly compares output against PRD; surfaces discrepancies — this step is not optional |

| Step | Actor | Action |
|---|---|---|
| 4. Direct | Human | Reviews identified issues; decides resolution; updates all reference documents |
| 5. Iterate | Both | Steps 3–4 repeat until no critical issues remain — in practice, ~3 cycles |
| 6. Advance | Human | Once verified, proceeds to the next plan step; each verified step becomes ground truth |

## 2.5 Status Labels

VIR uses consistent status labels across all document types to eliminate ambiguity:

| Status | Meaning | Used In |
|---|---|---|
| Open | Not yet addressed | Issues |
| In Progress | Work underway | Issues, Plans |
| Done / Resolved | Completed and validated | Issues, Bugs |
| Resolved (pending confirm.) | Fixed in code, awaiting production validation | Bugs |
| Partially resolved | Some aspects fixed, others remain | Bugs |
| Mitigated | Impact reduced but root cause not eliminated | Warnings |
| Superseded | No longer applicable due to architectural change | Bugs, Warnings |
| Accepted limitation | Intentional design choice, documented trade-off | Warnings |

## 2.6 Cross-Reference Conventions

VIR documents use precise cross-references to eliminate ambiguity between documents, files, and tracked items:

| Reference Type | Format | Example |
|---|---|---|
| File | module.py | db.py |
| File + line | module.py:123 | db.py:456 |
| Function | module.py → fn() | db.py → get_record() |
| PRD section | §N | §5 Config |
| Bug / Issue / Warn | B1, I3, W7 | Short ID for inline references |

© 2026 VIR Methodology

# 3. Document Types

VIR defines nine document types, each with a specific purpose. All templates are available in the repository's templates/ directory:

| Type | Purpose | Template |
|---|---|---|
| Root PRD | Axiomatic source of truth for the system | templates/PRD.md |
| Extension Spec | Detailed specification for a subsystem | templates/SPEC.md |
| Implementation Plan | Step-by-step build guide with acceptance tests | templates/PLAN.md |
| Bug Tracker | Production issues with full narratives | templates/BUGS.md |
| Issue Backlog | Feature gaps and improvements | templates/ISSUES.md |
| Warnings | Known limitations and mitigations | templates/WARNINGS.md |
| Integration Tests | Manual test suite with pass/fail criteria | templates/TESTS.md |
| Deployment Runbook | Production setup and operations guide | templates/ DEPLOYMENT.md |
| Agent Onboarding | AI agent context file (CLAUDE.md / AGENTS.md) | templates/AGENTS.md |

The minimum viable set scales with project complexity: start with Root PRD + CLAUDE.md; add BUGS.md and ISSUES.md for small projects; add Plans, Specs, WARNINGS.md and TESTS.md as the project grows; add DEPLOYMENT.md for production systems.

# 4. Working with AI Agents

## 4.1 The Agent Onboarding File (CLAUDE.md / AGENTS.md)

Every VIR project has an agent onboarding file — CLAUDE.md for Claude Code, or AGENTS.md for agent-agnostic setups. Claude Code automatically reads CLAUDE.md from the project root at the start of every conversation. It must be comprehensive enough that the agent can work effectively without reading every file in the repository. Required sections:

| Section | Why it matters |
|---|---|
| Project Summary | The agent needs to understand what the system does before touching any code |
| Repository Layout | Prevents files being created in wrong locations or missing existing code |
| Critical Conventions | Non-obvious rules the agent will violate without explicit instruction |

| Section | Why it matters |
|---|---|
| Module Reference | Saves the agent from reading every file to understand the codebase |
| Validation Steps | Tells the agent how to verify its own changes work |

End every CLAUDE.md with: "Trust these instructions. Only search the codebase if information here is incomplete or found to be incorrect." This directs the agent to use CLAUDE.md as its primary reference rather than re-reading the full codebase each session.

## 4.2 How Agents Use VIR Documents

| Agent Task | VIR Document | How It Helps |
|---|---|---|
| Understanding the system | CLAUDE.md + PRD | Authoritative architecture and conventions |
| Implementing a feature | Plan + Spec | Step-by-step guide with embedded tests |
| Fixing a bug | BUGS.md + WARNINGS.md | Prevents reintroducing known issues |
| Adding a component | PRD Extension Points | Consistent patterns for new code |
| Deploying changes | DEPLOYMENT.md | Exact commands and verification steps |
| Validating changes | TESTS.md | Runnable test commands with expected output |

## 4.3 Claude Code Workflows

### Building a New Feature

1. Write the spec first — create an extension spec in docs/spec/ from templates/SPEC.md
2. Create an implementation plan derived from the spec, using templates/PLAN.md — every step must have a runnable acceptance test
3. Execute the plan serially, one step at a time, running the acceptance test before proceeding
4. Reconcile: verify all acceptance tests pass, compare against the spec, update CLAUDE.md with new module documentation

### Fixing a Bug

1. Direct the agent to check BUGS.md for related known issues before investigating
2. After fixing, record the bug in BUGS.md with full narrative: root cause, solution, validation steps
3. Any significant decision made during the fix should be persisted to PRD.md and CLAUDE.md so future conversations inherit it

### Resuming Work Across Conversations

Each Claude Code conversation starts fresh — but VIR documents persist. To resume:

```
"I'm continuing work on the auth feature. Read docs/plan-auth.md
 to see where we left off. Steps 0–3 are complete. Continue from Step 4."
```

The agent reads the plan, sees the completed steps and their acceptance tests, and picks up exactly where the previous conversation ended. CLAUDE.md provides the architectural context; the plan provides the execution state.

### VIR Plans vs Claude Code Plan Mode

Claude Code has a built-in Plan Mode (Shift+Tab). VIR plans complement — not replace — this:

- Claude Code Plan Mode — lightweight, conversation-scoped, for immediate task planning
- VIR Plans — persistent .md files, cross-referenced to specs, with embedded runnable acceptance tests

For complex multi-step work spanning multiple sessions, use VIR plans. For quick within-session tasks, Claude Code's built-in planning is sufficient.

## 4.4 Key Rules for Agents

4. Trust the root PRD over any other document when conflicts arise
5. Read BUGS.md and WARNINGS.md before modifying affected components
6. Follow plans serially — do not skip steps or reconciliation
7. Run acceptance tests after completing each step
8. Use precise cross-references (file:line, §section) in all documentation updates
9. Update CLAUDE.md when adding new modules or changing conventions

## 4.5 Common Pitfalls

| Pitfall | Symptom | Fix |
|---|---|---|
| CLAUDE.md goes stale | Agent follows outdated conventions | After every significant change, ask the agent to update CLAUDE.md |
| PRD and code diverge | Agent doesn't know which source to trust | If code is intentionally different, update the PRD to match |
| Bug fixes without BUGS.md | Future conversation reintroduces the same bug | Always record bugs, even trivial ones, with root cause |
| Plans too vague or brittle | Steps are ambiguous or need constant revision | Each step: one-sentence goal, concrete deliverables, runnable test — no implementation prescriptions |

# 5. Canonical Project Structure

A mature VIR project follows this directory layout:

```
your-project/
├── CLAUDE.md                   # Agent onboarding — auto-read by Claude Code
├── docs/
│   ├── PRD.md                  # Root PRD — axiomatic source of truth
│   ├── BUGS.md                 # Bug tracker (VIR format)
│   ├── ISSUES.md               # Issue backlog (VIR format)
│   ├── WARNINGS.md             # Known limitations
│   ├── TESTS.md                # Integration test suite
│   ├── DEPLOYMENT.md           # Production runbook
│   ├── plan-feature-x.md       # Implementation plans (per feature)
│   └── spec/                   # Extension specifications
│       ├── PRD-auth.md
│       └── PRD-api.md
└── src/
```

Plans live in docs/ named by feature (plan-auth.md, plan-api-v2.md). Completed plans are kept — they are part of the project's history and can be referenced if issues emerge later. The spec/ subdirectory grows as the system is modularised.

# 6. Theoretical Basis: Why Convergence Occurs

## 4.1 Error Rate Reduction per Cycle

Let $\varepsilon_0$ represent the initial error rate of the LLM's output for a given task, and let $\alpha$ represent the fraction of errors caught by a single reconciliation cycle (the reconciliation efficiency). After n cycles, the residual error rate is:

$$\varepsilon_n = \varepsilon_0 \times (1 - \alpha)^n$$

For a reconciliation efficiency of $\alpha = 0.80$ (80% of errors caught per cycle) and three cycles, the residual error rate is $\varepsilon_0 \times 0.008$ — less than 1% of the original. The LLM does not need to be accurate; it needs to be correctable.

## 4.2 The Iterative Truth Stack

The deeper theoretical property of VIR is that each validated document layer becomes the foundation for generating the next. This creates an iterative truth stack:

```
Human insight  →  PRD.md        (axiomatic truth)
PRD.md         →  PLAN.md       (derived truth, validated against PRD)
PLAN.md + exec →  TESTS.md      (verified truth, empirically confirmed)
Live testing   →  BUGS.md       →  reconciliation  →  updated truth
```

Each layer inherits the determinism of the one above it, because it was generated by a probabilistic system but validated against a deterministic parent. The result is a specification stack that is provably internally consistent, built one validated layer at a time.

### 4.3 The Indispensability of Human Ground Truth

A critical property of VIR is that humans are not optional participants — they are the only valid source of ground truth at the axiomatic layer. An LLM asked to generate and then validate its own PRD is operating in a closed loop with no external reference, and will converge on internally consistent but potentially incorrect specifications.

Human practitioners provide three irreplaceable functions:

- They author the initial PRD — establishing the axioms
- They direct issue resolution after each reconciliation — maintaining correctness at the decision layer
- They validate deployment in the live environment — grounding the system in physical reality

All other tasks can be delegated to the LLM agent.

## 7. Structural Constraints of the Methodology

### 7.1 The Codebase Size Ceiling

The hallucination rate of an LLM is not constant — it increases with task complexity. As a codebase grows, the LLM must hold more state and more interdependencies within its context window. At some point, the error rate grows faster than the reconciliation mechanism can correct, and convergence fails.

The practical solution is modularisation. The system is decomposed into independently bounded subsystems, each with its own PRD, each small enough that the hallucination rate remains manageable. Modules interface through contracts — API specifications, data schemas — that are themselves absolute truth documents.

> *Implication: As LLM capability increases and hallucination rates decrease, the ceiling rises. The methodology scales with model improvement without structural change — larger systems become deliverable as models become more capable.*

### 7.2 The Serialisation Requirement

VIR cannot be parallelised across development steps, and this is not a limitation to be engineered around — it is a load-bearing property of the methodology. Each document layer is simultaneously an input and an output of the process. PLAN.md step N+1 cannot be developed in parallel with step N because step N's verified output is the ground truth that step N+1 builds on.

The analogy from construction is apt: the second floor cannot be poured before the first floor has cured. The sequential dependency is precisely what guarantees that each layer of truth is solid before the next layer depends on it.

# 8. Case Study: Production System Delivery in 4 Days

## 8.1 Context

Two non-developer practitioners — a technical lead and a researcher — applied VIR to deliver a complete production system from a working offline analysis codebase. Neither practitioner wrote code directly. All code was generated and executed by Claude Code (Opus 4.6), with GitLab for version control and PyCharm for inspection.

## 8.2 System Scope

The delivered system comprised:

- Real-time API integration layer for live data ingestion
- Analysis and control pipeline
- PostgreSQL logging database
- Frontend for system visualisation, authentication, and health monitoring
- Worker process for continuous operation
- Full deployment pipeline for an on-premises Linux server

## 8.3 Document Architecture in Practice

The PRD.md established that existing offline functionality was the baseline, and specified all requirements for the live environment. CLAUDE.md maintained agent working principles and context across frequent context window resets — a practical necessity at this codebase scale. PLAN.md decomposed delivery into discrete steps, each with embedded acceptance tests. BUGS.md captured live testing issues and fed them back through the reconciliation loop.

A key operational observation: context windows were dropped frequently due to codebase size. The document architecture functioned as durable external memory, allowing the agent to re-establish context from the specification stack rather than from the codebase itself.

## 8.4 Outcome

| Metric | Result |
|---|---|
| Delivery time | 4 days from specification to live production |
| Effort | ~20 person-hours per practitioner |
| Code authored | 0 lines written directly by practitioners |
| Reconciliation | ~3 cycles per PLAN.md step to reach zero critical issues |
| Live bugs | Resolved via BUGS.md reconciliation loop — no ad hoc fixes |

# 9. Broader Implications

## 9.1 Non-Developers as System Builders

VIR fundamentally changes who can deliver production software. The practitioner's role shifts from writing code to authoring specifications, directing reconciliation, and validating ground truth. These are skills of domain expertise, clarity of thought, and structured communication — not programming. The case study demonstrates that this is not theoretical.

## 9.2 Quality as an Emergent Property of Process

In conventional software development, quality is achieved through individual skill — experienced engineers write fewer bugs. In VIR, quality is an emergent property of the process architecture. The error rate of any individual LLM generation is largely irrelevant; what matters is the convergence rate of the reconciliation loop. This makes quality reproducible and process-dependent rather than talent-dependent.

## 9.3 The Scaling Horizon

As LLM capability improves, VIR's structural constraints relax. Lower hallucination rates mean the codebase size ceiling rises. Improved context windows reduce the frequency of context drops. Faster reconciliation cycles compress delivery timelines. The methodology does not need to change — it simply becomes more capable as the underlying model does.

# 10. Relationship to Prior Work

## 10.1 Specification-First Development

The principle that specifications should govern code has prior articulation. Spec-Driven Development (SDD) proposes that the entire development workflow reorganises around specifications as the central source of truth. VIR shares this foundation but extends it in a critical direction: where SDD treats specifications as a static input layer, VIR treats them as an iteratively generated and validated stack, where each layer is both derived from and certifies the one above it.

SELF-SPEC (2025) takes an automated approach, having the LLM author its own specification schema. The key distinction is that SELF-SPEC keeps specification generation within the LLM's closed loop — there is no external human-validated ground truth. VIR's axiomatic layer is explicitly human-authored, which is not a design choice but a theoretical necessity.

## 10.2 Iterative Feedback Loops in Code Generation

Iterative refinement via automated feedback is well-established. LLMLOOP (2025) demonstrates that wrapping LLM code generation in feedback loops progressively improves output quality. These approaches validate VIR's core convergence claim. The distinction is scope and control: these frameworks automate the correction signal; VIR's reconciliation step is human-directed. This is slower but provides a richer correction signal — one that can catch semantic divergence from intent that automated tools cannot detect.

© 2026 VIR Methodology

### 10.3 Original Contributions of VIR

Against this landscape, VIR makes three contributions not present in prior work:

1. The iterative truth stack — the theoretical framing of each validated document layer as the axiomatic foundation for the next, and the derivation of deterministic delivery from this layered structure
2. Derived structural constraints — the codebase size ceiling and serialisation requirement as necessary consequences of the methodology's correctness guarantee, derived from first principles
3. A practitioner methodology for non-developers — the first methodology designed specifically for domain-expert practitioners with no programming background, validated by a real-world production delivery

## 11. Conclusion

Verified Iterative Refinement is a methodology for achieving deterministic software delivery from probabilistic AI systems. It works not by making LLMs more reliable, but by engineering a feedback architecture around their unreliability — one that converges on correctness through structured reconciliation against human-authored ground truth.

The methodology rests on four principles: specification documents are boundary conditions, not guidelines; reconciliation is a required loop, not optional validation; every verified layer becomes the foundation for the next; and humans are the irreplaceable source of axiomatic truth.

In its mature form, VIR provides a complete operational discipline: a document authority hierarchy with a single root of truth; a five-phase development lifecycle with verification embedded at every step; a formal status and cross-reference system that eliminates ambiguity; and a scalable document set that grows with project complexity.

Two structural constraints follow necessarily from the architecture: a codebase size ceiling determined by hallucination rate and context window, and an irreducible serialisation requirement that preserves the integrity of the truth stack. Both are features, not limitations — they are the mechanisms by which the methodology's correctness guarantee holds.

The case study demonstrates that this is not abstract theory. Two non-developer practitioners delivered a complete production system in four days. The methodology is ready to be applied at scale.

*End of Document*

© 2026 VIR Methodology